Annexure No :

Practical 1

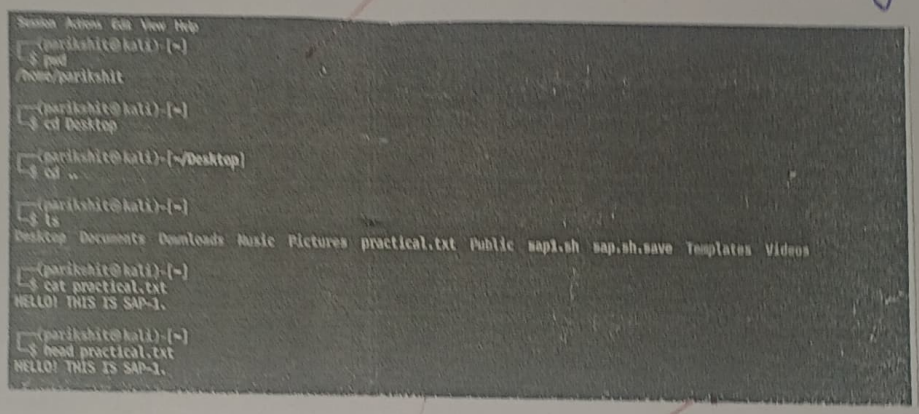
Aim: Study of Basic commands of Linux.

1. pwd - print working
- pwd prints the full pathname of the current working directory

2. cd - change
- It allows you to change your working directory.
- You use it to move around within the hierarchy of your file system

3. cd.. - Move up one directory

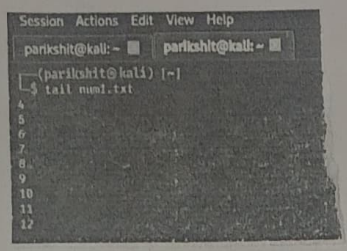
4. ls - List all the files and directories.
- List all files and folders in the current directory in the column format.

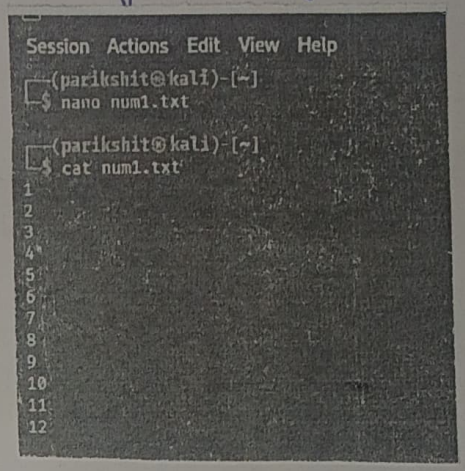


5. cat - catenate
- It reads the data from files and outputs their contents.
- It is the simplest way to display the contents of file at the command line.

6. head -
- By default prints the first 10 lines of each file to standard output. With more that file it proteats each set of output with a header identifying file name.

7. tail:
- tail command which prints the last few nos of lines of a certain file then terminates.



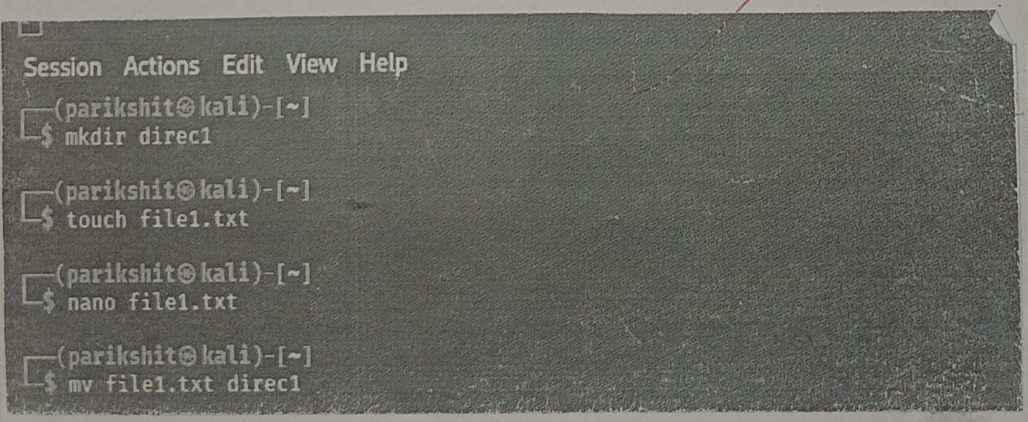


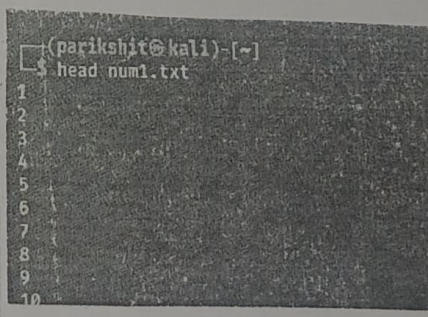
8. mv - Moving files
- The mv command lets you move a file from one directory location to another. It also lets you rename a file.

9. mkdir - make directory
- If the specified directory does not exist, mkdir creates it.
- More than one directory may be specified when calling mkdir.

10. cp - copy files
The cp command issued to make a copy of files and directories.

11. rmdir - remove directory
- The rmdir command is used to remove a directory that contains other files or directories.

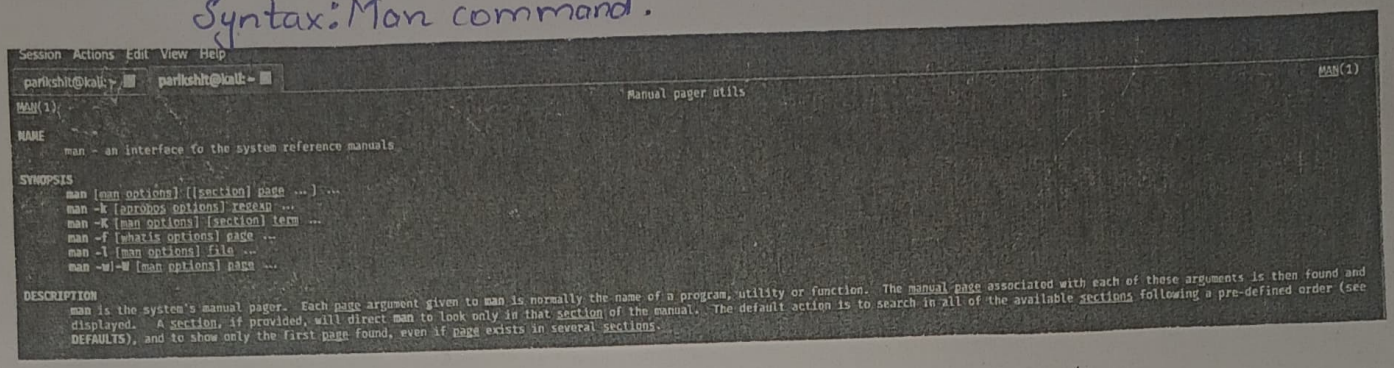




12. gedit: The gedit command is used to create and open a file.

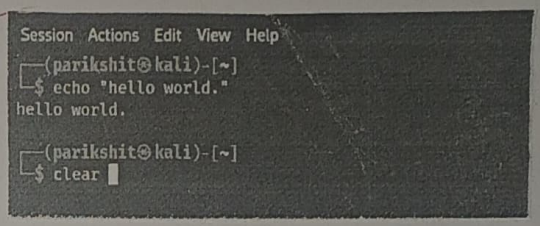
13. man: Displays on an online manual page or Manpage.

Syntax: Man command.



14. echo: Displays text on the screen.

Syntax: echo yourtext.

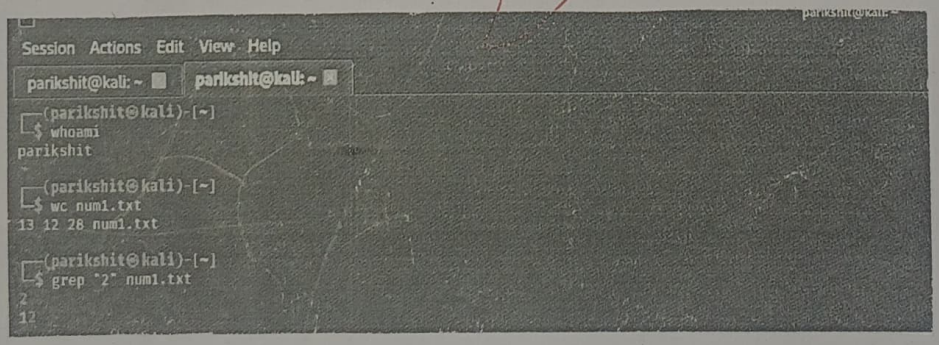


15. clear: Used to clear the screen.

Syntax: clear

16. whoami: whoami points the effective user ID. This commands prints the username associated with the current effective user Id.

Syntax: whoami [option]

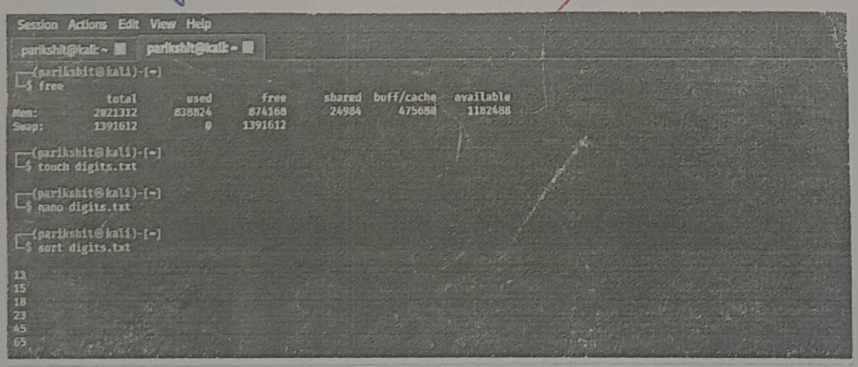


17. wc: wc(word count) command, can return the number of lines, words, and characters in a file.

18. grep: grep command uses a search term to look through a file.
Syntax: grep [option].... Pattern [file]...

19. free: Display RAM details in Linux Machine.
Syntax: free.

20. pipe(|): Pipe command is used to send output of one program as a input to another. Pipes "|" help combine 2 or more commands.
Syntax: command1 | command 2.



4/1/25

Practical-2

Aim: Study the basics of shell programming.

- The two prime components of operating system are -
  - Kernel
  - Shell
- A Kernel is at the nucleus of a computer. It makes the communication between hardware and software.
- A Shell in Linux O.S takes input from you in the from of commands, processes it and then gives the output.
- The shell issues a command prompt, where you can type your input, which is then executed when you hit the enter key. The result is thereafter displayed on the terminal.

→ Types of Shell: There are two main shells in Linux.

1. The Bourne Shell:

- The prompt for this shell is $ and its adv./derivatives are:
  - Korn Shell also known as sh.
  - Bourne Again shell is known as Bash.

2. The C Shell: The prompt for this shell is %, and its subcategories are:
   - The C Shell also known as csh.
   - Tops C shell also known as tcsh.

→ What is Shell Scripting?

Shell Scripting is writing a series of command for the shell to execute.

It can combine lengthy and repetitive sequence of commands into a single script.

→ Steps in creating a Shell Script:

1. Create a file using vi editors. Name script file with extension.sh
2. Start the script with #!/bin/bash.
3. Write some code.
4. Save the script files as filename.sh.
5. For executing the script type bash filename.sh.

Command 'ls' is executed when we execute the script filename.sh
To add a comment - # comment.

→ Shell variables:

- Variables store data in the form of characters and numbers.
- Shell variables are used to store information and they can by the shell only.

eg. variable = "Hello"
echo $variable.

echo is used to print the output or result.

```
Session  Actions  Edit  View  Help
parikshit@kali: ~   parikshit@kali: ~   parikshit@kali: ~
┌──(parikshit㉿kali)-[~]
└─$ touch practical2.sh

┌──(parikshit㉿kali)-[~]
└─$ nano practical2.sh

┌──(parikshit㉿kali)-[~]
└─$ chmod +x practical2.sh

┌──(parikshit㉿kali)-[~]
└─$ ./practical2.sh
Hello This is Parikshit
```

```
Session  Actions  Edit  View  Help
parikshit@kali: ~   parikshit@kali: ~   parikshit@kali: ~
  GNU nano 8.6
#!/bin/bash
echo "Hello This is Parikshit"
```

11/11/25

Parul University

Faculty of Engineering & Technology
Subject Name:
Subject Code:
B.Tech. ___Year____ Semester____

Annexure No :

Practical 3

Aim: Write a shell script to print the given numbers sum of all digits.

```
#!/bin/bash
#prompt user to enter a number.
echo "Enter a number:"
~~red~~ read number
Sum=0
while [$number-gt 0]
do
    mod=$((num%10))
    Sum=$((sum+~~digit~~ mod))
    num=$((num/10))
done
echo "Sum of digits: $Sum."
```

```
Session  Actions  Edit  View  Help
┌──(parikshit㉿kali)-[~]
└─$ cat practical3.sh
#!/bin/bash
echo "Enter a number"
read num
sum=0
while [ $num -gt 0 ]
do
mod=$((num%10))
sum=$((sum+mod))
num=$((num/10))
done
echo $sum

┌──(parikshit㉿kali)-[~]
└─$ ./practical3.sh
Enter a number
57
12
```

1/12/25

Enrollment No :

Page No :

Annexure No :

Practical 5

Aim: Write a shell script to check if the entered string is palindrome or not.

```
#!/bin/bash
# This script checks if the entered string is a palindrome or not
# Prompt the user for input
echo "Enter a string:"
read str
# Remove spaces and convert to lowercase for uniform comparison
str=$(echo $str | tr -d '[:spaces:]' | tr '[:upper:]' '[:lower:]')
# Reverse the string
rev_str=$(echo $str | rev)
# Check if the original string is equal to the reversed string
if ["$str" = "$rev_str"]; then
  echo "The string is a palindrome."
else
  echo "The string is not a palindrome."
fi
```

```
PARIKSHIT AWASTHI@DESKTOP-LQES02L MINGW64 ~/OneDrive/Desktop/OSLAB
$ ./Practical5.sh
Enter a string:
madam
The string is a palindrome.

PARIKSHIT AWASTHI@DESKTOP-LQES02L MINGW64 ~/OneDrive/Desktop/OSLAB
$ ./Practical5.sh
Enter a string:
hello
The string is not a palindrome.

PARIKSHIT AWASTHI@DESKTOP-LQES02L MINGW64 ~/OneDrive/Desktop/OSLAB
$
```

8/1/26

Annexure No :

Practical 6

Aim: Write a shell script to say Good Morning/Afternoon/Evening as you log in to the system.

```
#!/bin/bash
hour=$(date +"%H")
if [$hour -ge 5] && [$hour -lt 12]; then
echo "Good Morning!"
elif [$hour -ge 12] && [$hour -lt 17]; then
echo "Good Afternoon!"
else
echo "Good Evening!"
fi
```

```
PARIKSHIT AWASTHI@DESKTOP-LQES02L MINGW64 ~/OneDrive/Desktop/OSLAB
$ ./Practical6.sh
Good Afternoon!

PARIKSHIT AWASTHI@DESKTOP-LQES02L MINGW64 ~/OneDrive/Desktop/OSLAB
$
```

22/1/26

Annexure No :

Practical-7

Aim: Write a C program to create a child process.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
int main()
{
    pid_t pid;
    pid = fork();
    if (pid < 0) {
        perror("fork failed");
        return 1;
    }
    else if (pid == 0){
        printf("This is the child process. PID:%d, Parent PID:%d,
                Parent PID:%d\n", getpid(), getppid());
    }
    else {
        printf("This is the parent process. PID:%d, Child PID:%d\n",
                getpid(), pid);
    }
    return 0;
}
```

```
This is the parent process. PID: 852, Child PID: 856
This is the child process. PID: 856, Parent PID: 852


...Program finished with exit code 0
Press ENTER to exit console.
```

Annexure No :

Practical-9

Aim Printing the patters using for loop.

```
#include <stdio.h>
int main() {
    int i, j;
    for (i= 0; i <3; i++) {
        for (j= 0; j<4; j++) {
            printf(" * ");
        }
        printf("\n");
    }
    return 0;
}
```

Output:

```
* * * *
* * * *
* * * *
* * * *
```

Annexure No :

Practical-10

Aim- Shell script to determine whether a given file exists or not.

```
#!/bin/bash
echo "Enter the filename to check."
read filename
if [-e "$filename"]; then
    echo "The file '$filename exists".
else
    echo "The file '$filename' does not exist."
fi
```

Output:

```
Enter a filename to check:
Documents
the file 'Documents' exist.
```

12/12/26

Annexure No :

Practical - 11

Aim - Write a program for process creation using C.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
int main () {
    Pid-t pid;
    Pid = fork();
    if (Pid > 0){
      perror ("Jingala Failed");
      return 1;
    } else if (Pid==0) {
      printf ("This is the child process. PID %d, parent PID;
            %d \n", get Pid(), getPPid());
  } else {
      printf ("This is the parent process PID: %d, child PID;
            %d \n", get Pid(), pid);
    }
      return 0;
  }
```